

锂离子电池快充策略对寿命衰减的影响建模与优化

摘要

快充策略在缩短锂离子电池补能时间的同时会加速容量衰减，如何刻画快充策略与寿命衰减的定量关系并兼顾二者优化，是新能源汽车与储能系统的关键问题。本文基于 MIT-Stanford 锂离子电池循环老化数据集（49 块电池、9350 条循环记录、9 种两阶段快充策略），围绕快充策略参数（ C_1 、 Q_1 、 C_2 ）对寿命衰减的影响，建立了数据清洗、统计推断、寿命预测到多目标优化的完整流程，始终坚持可复现、留出验证与不确定性量化。

针对问题一：完成最小干预的数据清洗（修复 3 处异常），以函数型岭回归为主模型、样条混合与多项式混合为候选，按整块电池留一验证估计各策略 SOH 曲线。三种模型的策略排序完全一致，主模型 LOBO RMSE 为 0.004338；典型长寿命策略为 3_6C-80PER_3_6C 与 5C_67PER_4C，短寿命为 3_7C_31PER_5_9C 与 80PER_3_6C，但置换检验与 Holm 校正后两两间无显著差异。

针对问题二：引入低维应力坐标（总电流应力 J 与高 SOC 加权应力 H ）建立约束退化混合模型。全局置换检验显示策略间末段退化率差异显著（ $p = 5 \times 10^{-5}$ ），但选择校正后 $p = 0.065$ ，且删除极端策略后常数模型胜出——9 种策略的稀疏混杂设计不足以识别三个参数的独立因果效应，本文如实报告该边界。

针对问题三：经嵌套留一验证冻结轨迹岭回归模型 C_{ridge} ，其 $L = 150$ 策略等权 LOBO RMSE 为 0.000660；给出 9 块测试电池 151–200 循环预测与保守点区间，80% 寿命仅作情景外推（691.6–3653.0 循环）。

针对问题四：采用观测策略离散 Pareto 与整块电池 bootstrap。若优先缩短时间并接受约 0.17% 退化，推荐 5_3C_54PER_4C（ $C_1 = 5.3, Q_1 = 54\%, C_2 = 4.0$ ）：较长寿命策略缩短约 25% 时间、退化仅增约 0.12 个百分点。连续参数响应面因 oracle 压力测试失败被淘汰。

跨问题一致结论：充电策略标签与 SOH 退化相关，早期 SOH 动态是短期预测的主要信息，但稀疏混杂设计使结论须限制在已观测策略内，所有寿命终点均为外推估计。

关键字： 锂离子电池 快充策略 函数型岭回归 混合效应模型 留一验证 Pareto 优化

一、问题重述

1.1 问题背景

锂离子电池的快速充电能够显著缩短补能时间、提高使用便利性，但过高的充电电流会导致电池内部极化增强、热效应加剧与副反应加速，从而加快容量衰减、缩短电池寿命^[1]。电池寿命通常以健康状态 SOH 表征，定义为当前可用容量与初始容量之比，本题统一以 80% SOH 作为寿命终止阈值。

影响电池寿命的两个主要指标为充电倍率（C-rate）与荷电状态 SOC。不同 SOC 区间内电池对充电电流的敏感程度不同：在低 SOC 区间采用较大电流可能有助于缩短充电时间，而在中高 SOC 区间继续采用高倍率充电则可能加剧老化。本题采用 MIT-Stanford 公开数据集（49 块 A123 18650 磷酸铁锂电池，额定容量 1.1 Ah），各电池放电策略一致，差异体现在两阶段快充策略：先以 C_1 倍率从 0% 充电至 Q_1 ，再以 C_2 倍率从 Q_1 充电至 80%，最后以 1C 恒流-恒压（CC-CV）充至满电。

1.2 问题要求

- (1) 整理数据，分析不同快充策略下的寿命分布，提取并对比典型长寿命与短寿命策略；
- (2) 分析充电策略参数（ C_1, Q_1, C_2 ）对寿命衰减的影响，检验差异显著性及参数影响程度；
- (3) 基于 9 块测试电池截至第 150 循环的数据建立寿命预测模型，预测第 151–200 循环 SOH 并外推寿命，评价精度及早期数据长度的影响；
- (4) 设计兼顾充电时间与寿命衰减的策略优化方法，给出推荐策略并与典型策略对比。

二、问题分析

2.1 数据特点

本数据集有三个关键特征：（1）9350 条循环记录是 49 块电池的重复测量，观测非独立，显著性检验与交叉验证必须按整块电池划分；（2）仅有 9 种策略， C_1, Q_1, C_2 联合变化，参数坐标高度相关，且存在新旧结构/数据批次混杂；（3）数据只观测至第 200 循环，SOH 最低约 0.95，无任何电池达到 80% 终点，因此寿命终点只能外推^[2]。

2.2 各问题分析

问题一是探索性建模。用函数型/混合效应模型估计各策略 SOH 曲线，按整块电池留一验证选模，以 bootstrap 量化不确定性，识别典型长/短寿命策略。

问题二是统计推断。核心难点是稀疏混杂设计下参数效应不可识别。思路是构造物理启发的低维应力坐标，用约束退化混合模型刻画退化，用置换检验与选择校正检验给出稳健的显著性结论 [2, 7]。

问题三是预测。从早期循环提取轨迹特征，比较六类模型，用嵌套留一验证选择并冻结主模型，预测测试电池未来轨迹并给出区间；寿命终点作为情景外推 [2, 7]。

问题四是优化。在 9 个观测策略上做离散 Pareto 分析，用整块电池 bootstrap 评估前沿稳定性，给出条件约束下的推荐；连续参数搜索因设计不足被谨慎淘汰 [2, 7]。

三、 模型假设

- (1) 假设电池容量测量误差为随机噪声，单次异常尖峰不代表真实寿命改善，可作为数据质量异常修复；
- (2) 假设各电池放电策略一致，电池间差异来源于充电策略与个体制造差异；
- (3) 假设 SOH 在观测窗口内近似平滑，可用函数型/混合效应模型描述，且平均 SOH 随循环单调下降；
- (4) 假设充电时间主要由两阶段恒流决定，恒压尾段与结构/批次影响作为经验项；
- (5) 假设推断单位是整块电池而非循环记录，显著性检验与重采样均按电池/策略聚类；
- (6) 忽略温度、内阻的精细机理，仅将其作为观测协变量 [7]。

四、 符号说明

表 1 符号说明

符号	含义
C_1	第一阶段充电倍率（低 SOC 区， $0 \sim Q_1$ ）
Q_1, q	阶段切换 SOC（%）； $q = Q_1/100$
C_2	第二阶段充电倍率（中高 SOC 区， $Q_1 \sim 80\%$ ）
J	总电流应力 $qC_1 + (0.8 - q)C_2$
H	高 SOC 加权应力 $\frac{1}{2}[C_1q^2 + C_2(0.8^2 - q^2)]$
T_0	理想恒流充电时间（min）
$SOH_{i,t}$	第 i 块电池第 t 循环的健康状态
r_i	第 i 块电池的退化率（对数尺度）

D_p	策略 p 的 200 循环退化量
T_p	策略 p 的平均充电时间

五、数据预处理

5.1 原始数据核对

原始数据含电池级摘要表（49 行）与循环级时序表（9350 行）。核对确认 40 块完整电池包含第 1–200 循环，9 块测试电池（ $prediction_test = 1$ ）仅含第 1–150 循环；共 9 种充电策略。

5.2 异常识别与可审计修复

对循环数据逐电池检测局部异常：容量采用 ± 5 循环邻域的稳健 z 分数与相对偏差判定，内阻以 $IR \leq 0$ 判定为物理无效。修复采用相邻有效循环线性插值，共 3 处（表 ??），原始值与修复标记全部保留。

表 2 数据清洗修复记录

电池	循环	变量	原始值	修复值	依据
1	12	容量/Ah	1.539054	1.074304	稳健 z 与相对偏差双超限
2	12	内阻/ Ω	0	0.016869	内阻必须为正
3	12	内阻/ Ω	0	0.016683	内阻必须为正

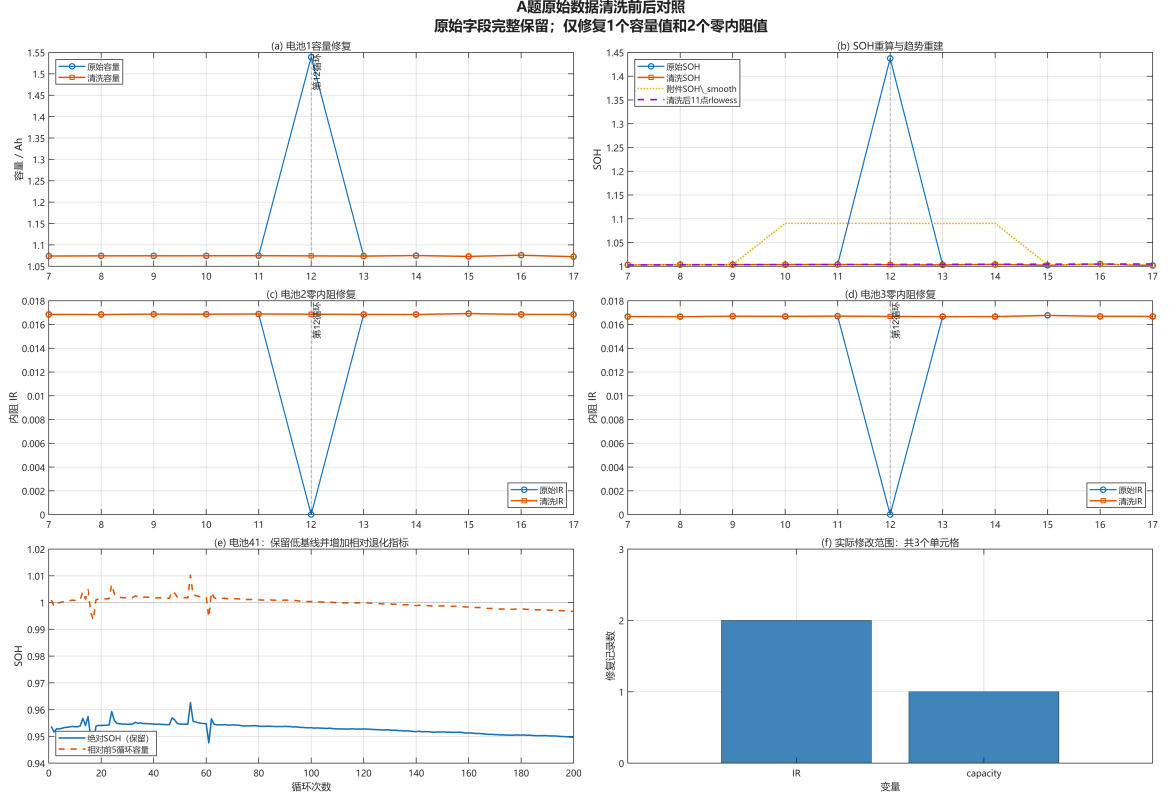


图 1 数据清洗前后对照（容量尖峰与零内阻修复）

电池 41 存在初始 SOH 基线偏低（约 0.953），判断为批次/定义差异而非单点异常，予以保留并增加相对 SOH 辅助指标；策略 $80PER_3_6C$ 的 C_1 缺失（单阶段策略），不猜测填补，仅进入类别比较。清洗后保留全部 49 块电池、9350 条记录。

六、问题一模型的建立与求解

6.1 函数型混合模型

将每块电池的 SOH 视为关于循环数的函数曲线，建立函数型岭回归主模型

$$SOH_{i,t} = f(t) + f_{p(i)}(t) + b_i + \varepsilon_{i,t}, \quad (1)$$

其中 $f(t)$ 为总体非线性退化趋势， $f_{p(i)}(t)$ 为策略 $p(i)$ 的偏离， b_i 为电池个体差异， $\varepsilon_{i,t}$ 为噪声。以样条混合（spline_mixed）与多项式混合（polynomial_mixed）为候选对照。模型用 40 块完整电池按整块电池留一验证（LOBO）评价，策略级指标以整块电池 bootstrap 估计标准误与 95% 区间。

6.2 模型选择

三个候选模型的 LOBO RMSE 极其接近（表 ??），函数型岭以 0.004338 略优，选为主模型；且三模型对第 200 循环 SOH 的策略排序完全一致，说明排序稳健。

表 3 问题一候选模型比较（LOBO）

模型	平均电池 RMSE	平均电池 MAE	最差策略 RMSE	选定
polynomial_mixed	0.004394	0.004071	0.018335	否
spline_mixed	0.004342	0.004036	0.018332	否
functional_ridge	0.004338	0.004033	0.018332	是

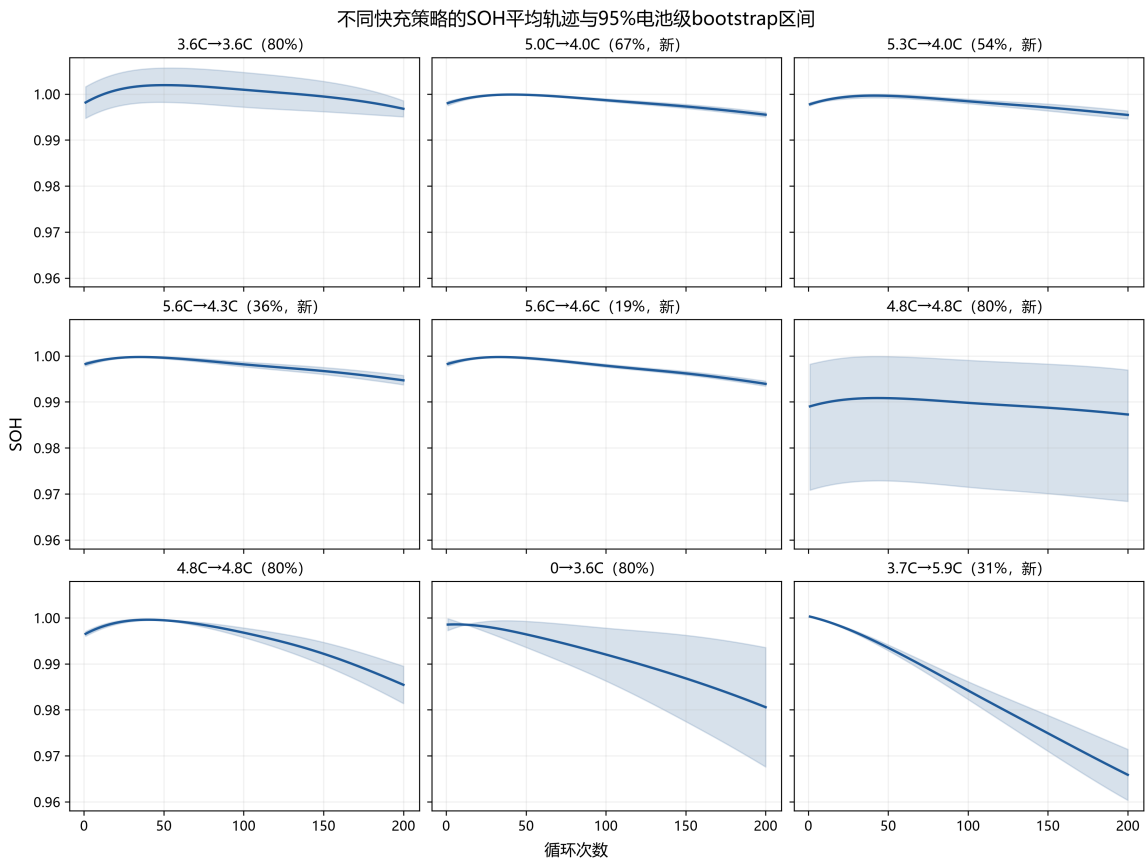


图 2 各策略 SOH 曲线（函数型岭主模型拟合）

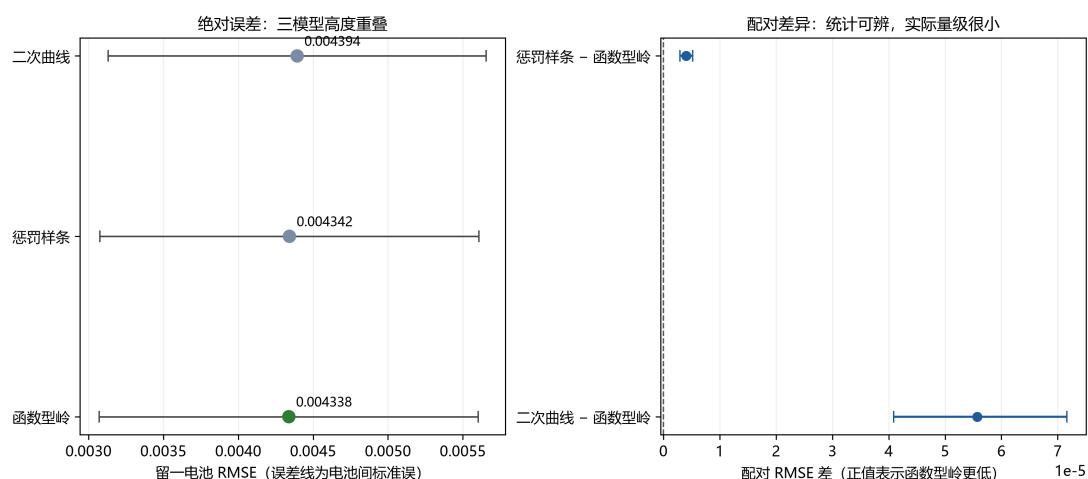


图 3 三候选模型 LOBO 误差对比

6.3 策略寿命分布与典型策略识别

各策略第 200 循环 SOH 的 bootstrap 点估计与区间见表 ?? (部分策略)。典型长寿命策略为 3_6C-80PER_3_6C (SOH200 = 0.9968) 与 5C_67PER_4C (0.9955)；典型短寿命策略为 3_7C_31PER_5_9C (0.9659, 最差)、80PER_3_6C (0.9806) 与 4_8C_80PER_4_8C (0.9854)。

表 4 各策略 SOH@200 点估计与 bootstrap 区间

策略	SOH200 点估计	95% 区间	充电时间/min
3_6C-80PER_3_6C	0.9968	[0.9951, 0.9985]	13.388
5C_67PER_4C	0.9955	[0.9950, 0.9960]	10.159
5_3C_54PER_4C	0.9954	[0.9946, 0.9963]	10.158
4_8C_80PER_4_8C	0.9854	[0.9814, 0.9895]	13.082
80PER_3_6C	0.9806	[0.9676, 0.9935]	14.280
3_7C_31PER_5_9C	0.9659	[0.9603, 0.9714]	10.852

6.4 两两比较与寿命终点边界

精确置换检验配合 Holm 校正后，36 组 SOH200 两两比较中没有任何策略对显著——9 种策略每类仅 3–8 块电池，统计功效不足，故排序仅作为描述性结论。此外，49 块电池均未观测到 80% SOH 终点，局部线性外推的 L_{80} 只能作为未验证的外推代理，不作为寿命标签。

七、问题二模型的建立与求解

7.1 低维应力坐标

设 $q = Q_1/100$ ，两阶段倍率函数为

$$c(s) = \begin{cases} C_1, & 0 \leq s \leq q, \\ C_2, & q < s \leq 0.8, \end{cases} \quad (2)$$

据此定义理想恒流充电时间、总电流应力与高 SOC 加权应力：

$$T_0 = 60 \left(\frac{q}{C_1} + \frac{0.8 - q}{C_2} \right), \quad J = qC_1 + (0.8 - q)C_2, \quad H = \frac{1}{2} [C_1 q^2 + C_2 (0.8^2 - q^2)]. \quad (3)$$

J 描述主要充电区间的总体倍率暴露， H 对高 SOC 区间赋予更高权重。二者相关系数为 0.87， T_0 与 J 相关系数为 -0.984，三者不能同时进入正式模型。

7.2 约束退化混合模型

以第 151–200 循环的末段退化率（对数尺度）为响应，建立

$$y_{it} = a_i - r_i x_t - k x_t^2 + \varepsilon_{it}, \quad x_t = t/200, \quad (4)$$

$$\log r_i = \beta_0 + \beta_J \tilde{J}_{p(i)} + \beta_H \tilde{H}_{p(i)} + \beta_S S_{p(i)} + u_i, \quad (5)$$

其中 $r_i > 0$ 、 $k \geq 0$ 保证平均 SOH 单调下降， u_i 为同策略电池差异， S 为“新结构/数据批次”联合标签（不作纯结构因果解释）。模型选择使用留一策略验证、AICc 与系数稳定性。

7.3 统计显著性

对策略间末段退化率做全局置换检验： $F = 12.50$ 、置换 $p = 5 \times 10^{-5}$ （20000 次置换），表明策略标签与末段退化存在全局关联。高 SOC 倍率暴露（ H ）与退化方向一致，但选择校正后 $p = 0.065$ 未达显著，且删除极端策略后常数模型胜出（图 ??）。

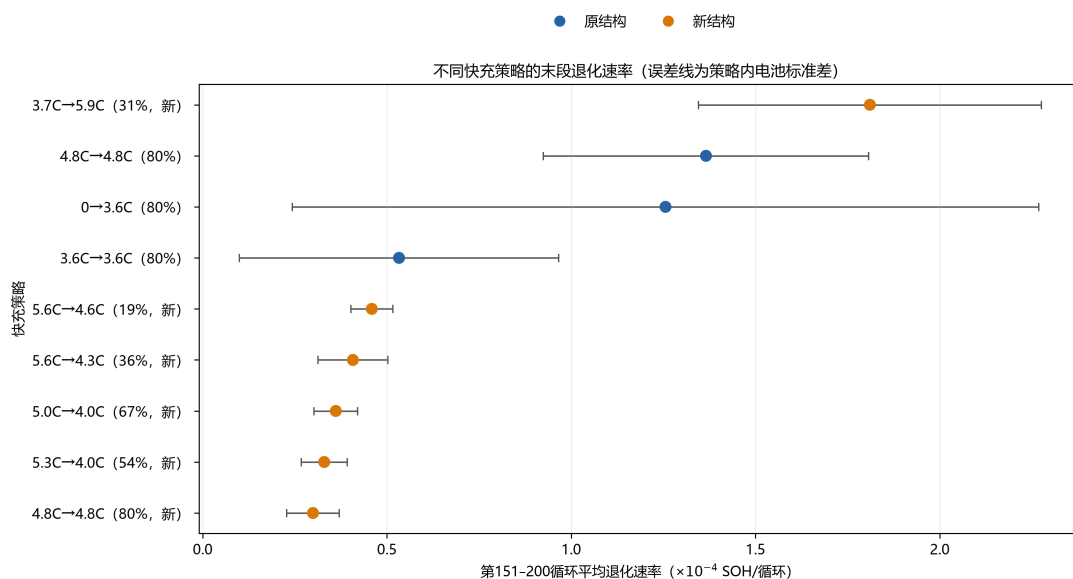


图 4 各策略末段（151–200 循环）退化率

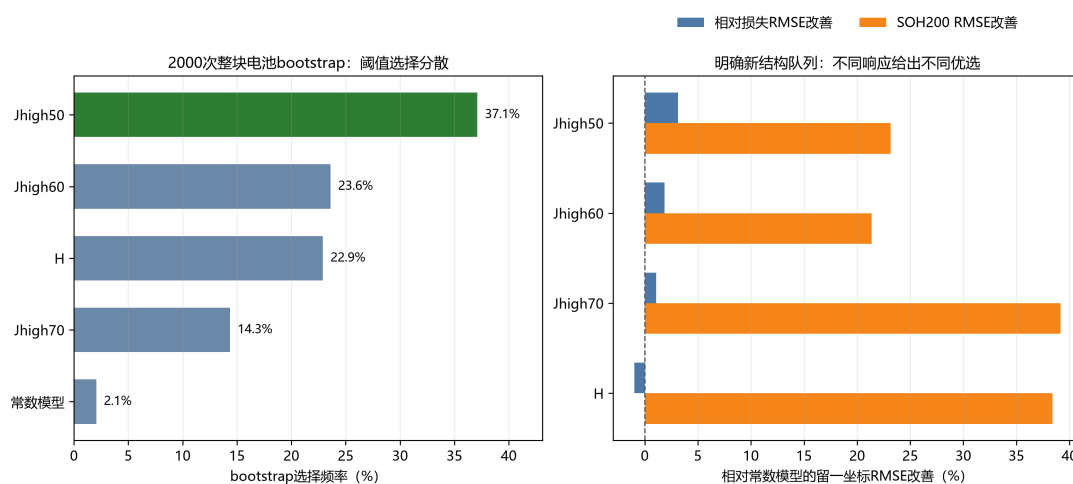


图 5 模型系数稳定性与选择校正

7.4 结构与批次混杂

两个具有相同 (4.8, 0.8, 4.8) 参数的策略分属旧/新结构，末段退化率分别为 0.000137 与 0.000030，配对置换 $p = 0.048$ ；但结构、数据批次与策略参数完全混杂，无法将此差异干净地归因于“结构”。

7.5 小问结论

综上，回答五个小问：(1) 策略间末段退化存在全局显著差异；(2–3) C_1, Q_1, C_2 与退化的关联方向一致（高 SOC 高倍率加剧退化），但受稀疏混杂设计限制，不能识别三个参数的独立因果效应；(4) 不同 SOC 区间高倍率的差异化作用方向与机理预期一致，

但证据强度不足以形成因果断言；(5) 结论的机制可解释为高 SOC 下高倍率促进锂析出与极化加剧，其局限性在于设计混杂与样本量不足 [7]。

八、问题三模型的建立与求解

8.1 特征与候选模型

从早期循环提取 SOH 轨迹特征，比较六类候选模型：轨迹岭回归 (C_{ridge})、线性外推、持久化、幂律、策略迁移与机器学习基线等。以 40 块完整电池截断至 $L \in \{50, 100, 150\}$ 循环做外层留一电池、折内调参、嵌套选族。

8.2 模型选择与预测

综合多长度后冻结 C_{ridge} 。其固定模型族在 $L = 150$ 的策略等权 LOBO RMSE 为 0.000660，含调参选族的嵌套流程为 0.000709；随早期长度增加误差单调下降（图 ??）。 $L = 150$ 消融显示仅用动态特征 RMSE 0.000645 优于完整模型，说明策略特征在该窗口不提供增量精度。

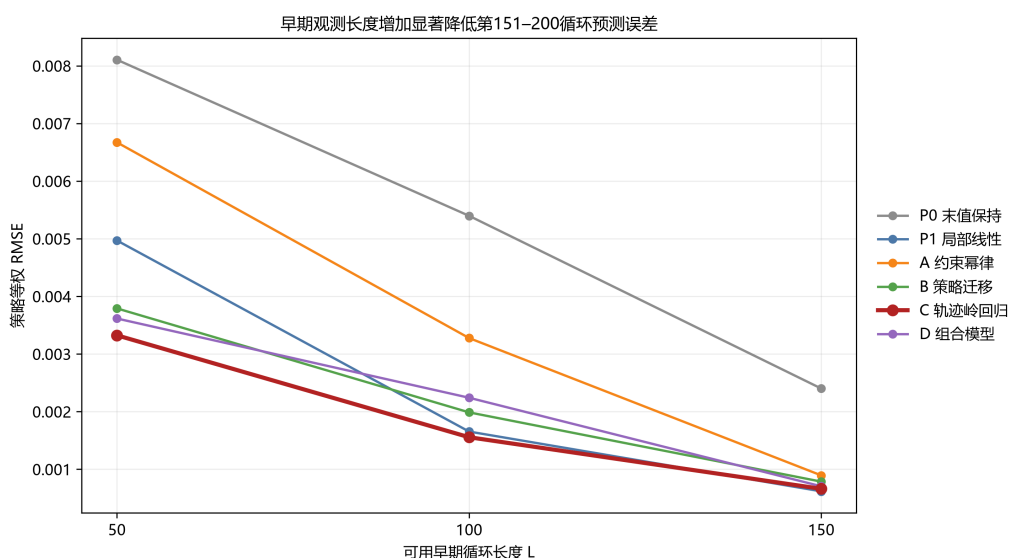


图 6 六候选模型随早期数据长度的 LOBO RMSE

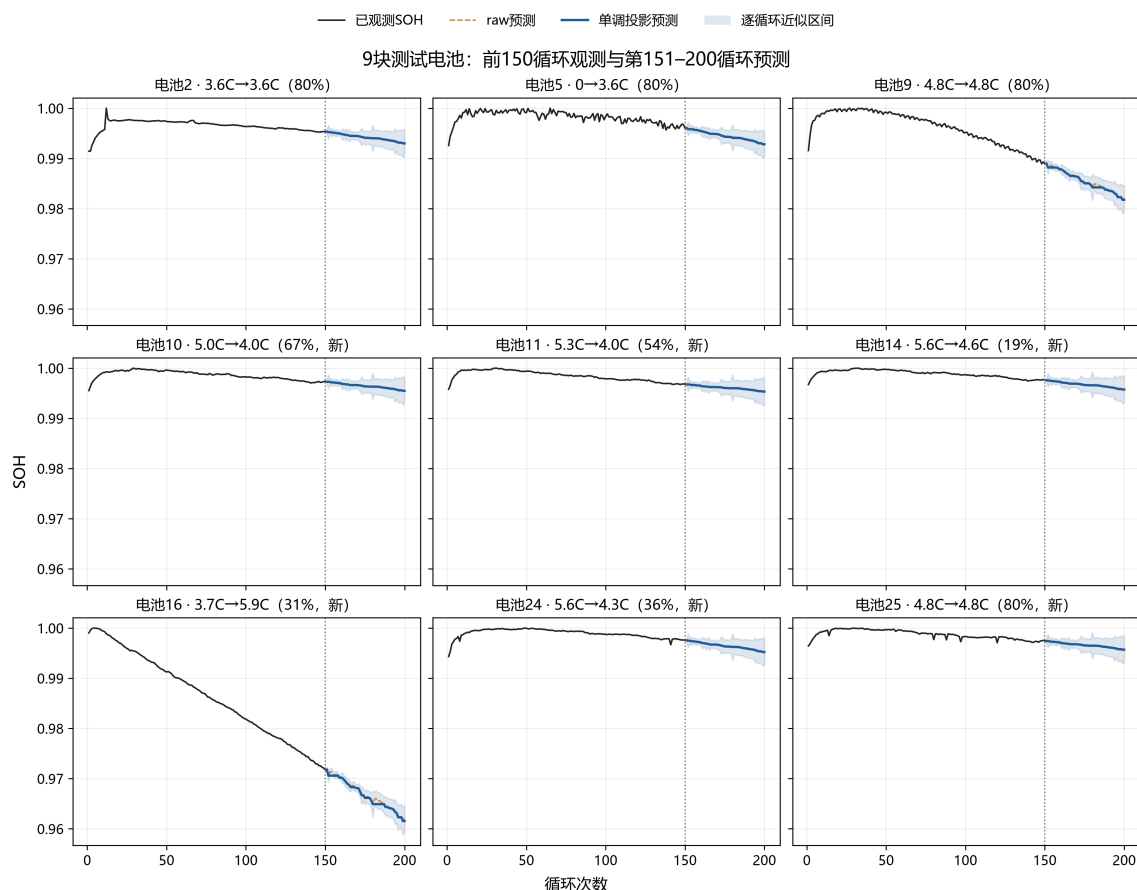


图 7 9 块测试电池 151–200 循环预测（实线单调投影、阴影保守点区间）

8.3 测试电池预测

9 块测试电池的预测 SOH₂₀₀ 与保守点区间见表 ??。区间由冻结模型族在 40 块训练电池上的外层交叉拟合绝对残差的 97.5% 经验分位逐循环校准，为偏保守的逐循环点区间，不构成整条轨迹联合区间或独立测试覆盖率。

表 5 9 块测试电池预测（部分）

电池	策略	预测 SOH ₂₀₀	保守点区间	默认情景 T_{80}
2	3_6C-80PER_3_6C	0.993053	[0.9904, 0.9957]	774.0
10	5C_67PER_4C	0.995516	[0.9929, 0.9981]	1391.1
16	3_7C_31PER_5_9C	0.962027	[0.9594, 0.9647]	916.1
25	4_8C_80PER_4_8C	0.995728	[0.9931, 0.9984]	1804.8

8.4 80% 寿命外推的边界

80% 寿命终点 (T_{80}) 对拟合起点与 raw/projected 口径高度敏感 (图 ??)，有限情景值范围为 691.6–3653.0 循环。由于 49 块电池均未观测到 80% SOH， T_{80} 只能作为模型

情景交点，不具有实证寿命精度，论文将其与短期 LOBO 误差分开陈述。

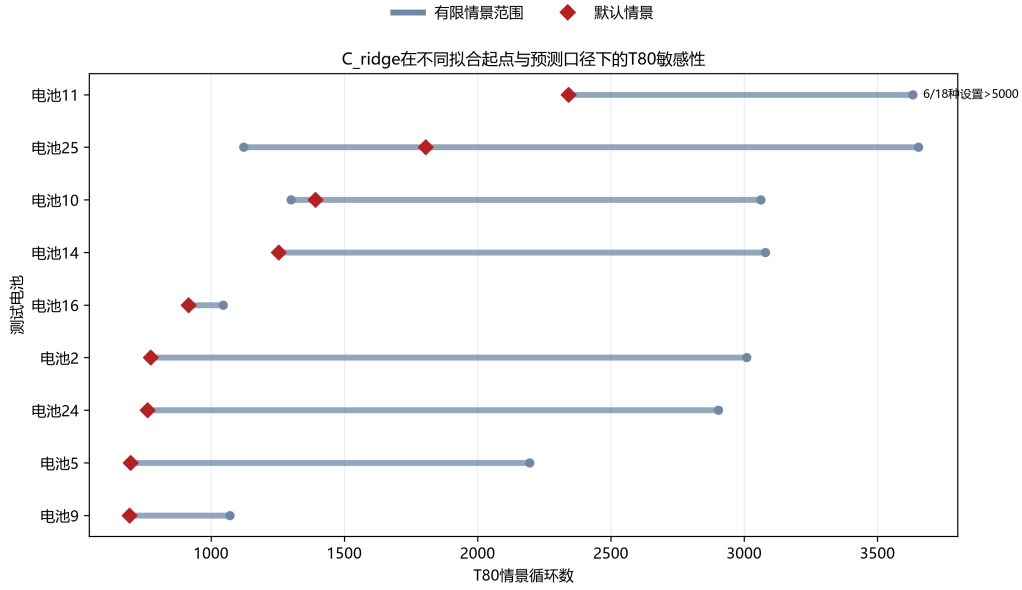


图 8 T_{80} 情景外推敏感性（红菱形为默认情景）

九、问题四模型的建立与求解

9.1 离散 Pareto 主模型 (M0)

对策略 p 定义充电时间与退化量

$$T_p = \text{mean}_{i \in p}(\text{chargetime}_i), \quad D_p = \text{mean}_{i \in p} \left(1 - \text{SOH}_{i,200}/b_i \right), \quad (6)$$

其中 b_i 为电池基线容量。若不存在策略 r 使 $T_r \leq T_p$ 且 $D_r \leq D_p$ (至少一项严格)，则 p 位于 Pareto 前沿。时间与退化用整块电池 bootstrap 量化不确定性。理论恒流时间 T_0 仅作解释量，实测时间含恒压尾段与结构/批次影响。

9.2 Pareto 前沿与推荐

点估计 Pareto 前沿含 3 个策略（表 ??），bootstrap 显示前沿成员不稳定。5_3C_54PER_4C 的 Pareto 进入频率最高（0.892），3_6C 基准为 0.740，4_8C 新结构为 0.514。

表 6 点估计 Pareto 前沿（40 块完整电池）

策略	n	时间/min	200 循环退化	末段衰减率	Pareto 频率
3_6C-80PER_3_6C	2	13.382	0.000428	0.0000500	0.740
4_8C_80PER_4_8C（新）	5	11.039	0.001436	0.0000277	0.514
5_3C_54PER_4C	7	10.043	0.001615	0.0000315	0.892

表 7 推荐策略与典型策略对比

策略	时间/min	200 循环退化	较长寿命时间差	较长寿命退化差
3_6C-80PER_3_6C（典型长寿命）	13.382	0.000428	0	0
5_3C_54PER_4C（推荐）	10.043	0.001615	-3.340	+0.001187
3_7C_31PER_5_9C（典型短寿命）	10.134	0.033617	-3.249	+0.033189

推荐 5_3C_54PER_4C ($C_1 = 5.3, Q_1 = 54\%, C_2 = 4.0$): 若应用要求优先缩短时间并接受约 0.17% 的 200 循环相对退化, 该策略是近似并列最快组（四个最快新结构策略仅相差不足 0.03 s）中退化最低者。相对典型长寿命策略缩短约 25% 时间、退化仅增约 0.12 个百分点；相对典型短寿命策略更快 0.09 min 且退化低约 0.032。其机理是低 SOC 段使用较高倍率、较早切换至 4C, 形成观测数据支持的时间-退化折中。注意它在时间上仅属于四策略近似并列组, 不能宣称显著更快。

9.3 连续响应面的淘汰（M1）

受限连续响应面（单 J 岭模型）在 oracle 坐标压力测试中平均 RMSE 为 0.015827, 相对常数基线改善为 -0.006806, 7 折中 6 折未优于常数基线, 因此连续三参数优化被正式淘汰, 正式答案退回离散 Pareto。

9.4 合理性、适用范围与外推风险

推荐合理性来自四条证据：位于点估计 Pareto 前沿、bootstrap Pareto 频率最高、两种时间口径下前沿不变、SOH200 与末段斜率方向一致。结论严格限定在 9 个已观测策略范围内, 不宣称 C_1, Q_1, C_2 的独立因果最优值, 不外推到未观测倍率、SOC 区间或新电芯结构^[2, 3]。

十、灵敏度分析

10.1 清洗规则敏感性

固定 3 处修复但改变局部趋势窗口（7/11/15 点）后，策略排序不变；保留异常原始值仅影响个别策略指标。

10.2 响应口径敏感性

Q1 以 SOH200、前 200 循环损失、曲线面积与末段斜率多口径检验，策略排序高度一致；Q4 的退化代理允许为负（受基线噪声影响），如 3.6C 策略退化均值为 0.000428、bootstrap 区间下界为 -0.0015 ，已接近测量噪声地板。

10.3 时间口径敏感性

Q4 采用 40 块完整电池逐循环充电时间均值为主口径，官方汇总口径作敏感性审计；两口径在部分策略差异明显（如旧结构 4.8C 为 10.45 与 13.08 min），但 Pareto 前沿与权重选择不变。

10.4 权重敏感性

Q4 权重扫描中，全 9 策略 min-max 缩放下 $\lambda = 0.1 \sim 1.0$ 选 5.3C；仅 3 个 Pareto 点缩放下 $\lambda = 0 \sim 0.5$ 选 3.6C、 $\lambda = 0.6 \sim 1.0$ 选 5.3C，说明膝点依赖标准化集合，故正式结论以 Pareto 与条件约束为主，权重仅描述偏好敏感性。

十一、模型评价与推广

11.1 模型优点

- 数据清洗最小干预、全程审计留痕，修复决策与人工核对一致；
- 始终以整块电池为推断单位，正确处理重复测量与聚类重采样；
- 多模型互证（函数型岭/样条/多项式），并用置换检验、选择校正、bootstrap 量化不确定性；
- 诚实地报告了设计混杂导致的因果识别边界，不强行宣称参数独立效应。

11.2 模型不足

- 9 种策略的稀疏混杂设计使 C_1, Q_1, C_2 独立因果效应不可识别；
- 无真实 80% SOH 终点，寿命终点均为外推；
- 2 电池策略（如 3.6C）的分布组合极少，bootstrap 只能作粗粒度证据。

11.3 推广方向

可引入层次贝叶斯模型刻画策略总体与电池个体差异并部分汇聚^[2,3]，在闭环优化框架中结合贝叶斯优化与不确定性进行实验设计^[2]，并针对末段斜率反映的退化反转做更细致的机理分析^[2]。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于【代码调试、语言润色、数据处理与建模方案讨论】，详细使用情况见支撑材料中“AI 工具使用详情.pdf”。核心建模思路、模型选择、结果分析与关键结论均由参赛队独立完成，AI 生成内容已经过逐项人工审查与核实。

参考文献

- [1] Severson K A, Attia P M, Jin N, et al. Data-driven prediction of battery cycle life before capacity degradation[J]. Nature Energy, 2019, 4(5): 383-391.
- [2] Attia P M, Grover A, Jin N, et al. Closed-loop optimization of fast-charging protocols for batteries with machine learning[J]. Nature, 2020, 578(7795): 397-402.
- [3] Jiang B, Gent W E, Mohr F, et al. Bayesian learning for rapid prediction of lithium-ion battery-cycling protocols[J]. Joule, 2021, 5(12): 3187-3203.
- [4] Keil P, Jossen A. Charging protocols for lithium-ion batteries and their impact on cycle life[J]. Journal of Energy Storage, 2016, 6: 125-141.
- [5] Schuster S F, Bach T C, Fleder E, et al. Nonlinear aging characteristics of lithium-ion cells under different operational conditions[J]. Journal of Energy Storage, 2015, 1: 44-53.
- [6] Saxena S, Xing Y, Kwon D, et al. Accelerated degradation model for C-rate loading of lithium-ion batteries[J]. International Journal of Electrical Power & Energy Systems, 2019, 107: 438-445.
- [7] Richardson R R, Osborne M A, Howey D A. Gaussian process regression for forecasting battery state of health[J]. Journal of Power Sources, 2017, 357: 209-219.
- [8] Liu K, Hu X, Wei Z, et al. Modified Gaussian process regression models for cyclic capacity prediction of lithium-ion batteries[J]. IEEE Transactions on Transportation Electrification, 2019, 5(4): 1225-1236.

- [9] Zhou Z, Howey D A. Bayesian hierarchical modelling for battery lifetime early prediction[J]. IFAC-PapersOnLine, 2023, 56(2): 6117-6123.
- [10] Geslin A, van Vlijmen B, Cui X, et al. Selecting the appropriate features in battery lifetime predictions[J]. Joule, 2023, 7(9): 1956-1965.

附录 A 支撑材料文件列表

表 8 支撑材料文件列表

类别	路径	说明
数据	data/processed/q1_cleaned/*.csv	清洗后摘要表与循环表
结果	result/q1/paper/*.csv	问题一结果与结论
结果	result/q2/03_formal_validation/	问题二正式验证
结果	result/q3/03_final_predictions/	问题三最终预测
结果	result/q4/02_full_validation/	问题四全量验证与推荐
代码	src/q1_models/ 等	各问模型核心源码
代码	scripts/q1-q4/	各问实验入口脚本

附录 B 核心算法代码

以下为核心模型源码（完整可运行代码见支撑材料，运行顺序见各 scripts 入口）。

```

1  """Numerical core for the three comparable Question 1 curve models."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from typing import Iterable
7
8  import numpy as np
9  import pandas as pd
10
11
12  MODEL_POLYNOMIAL = "polynomial_mixed"
13  MODEL_SPLINE = "spline_mixed"
14  MODEL_FUNCTIONAL = "functional_ridge"
15  MODEL_TYPES = (MODEL_POLYNOMIAL, MODEL_SPLINE, MODEL_FUNCTIONAL)
16
17
18  @dataclass(frozen=True)
19  class ModelConfig:
20      lambda_random: float = 0.1
21      lambda_curve: float = 0.01
22
23
24  @dataclass

```



```

25 class PopulationCurveModel:
26     model_type: str
27     config: ModelConfig
28     policy_names: tuple[str, ...]
29     fixed_coef: np.ndarray
30     battery_ids: np.ndarray
31     random_coef: np.ndarray | None = None
32     cell_coef: np.ndarray | None = None
33     cell_policy: np.ndarray | None = None
34
35     def predict(self, policy: str | Iterable[str], cycle: Iterable[float]) -> np.ndarray:
36         cycles = np.asarray(cycle, dtype=float).reshape(-1)
37         policies = np.asarray([policy] * len(cycles) if isinstance(policy, str) else policy, dtype=str)
38         if len(policies) != len(cycles):
39             raise ValueError("policy and cycle must have the same length")
40         basis = make_basis(self.model_type, cycles)
41         output = np.full(len(cycles), np.nan, dtype=float)
42         policy_to_index = {name: i for i, name in enumerate(self.policy_names)}
43         for name in np.unique(policies):
44             if name not in policy_to_index:
45                 raise KeyError(f"policy absent from training data: {name}")
46             use = policies == name
47             output[use] = basis[use] @ self.fixed_coef[policy_to_index[name]]
48         return output
49
50
51     def make_basis(model_type: str, cycle: Iterable[float]) -> np.ndarray:
52         """Return the explicitly specified cycle basis, using  $x=t/200$ ."""
53         x = np.asarray(cycle, dtype=float).reshape(-1) / 200.0
54         if model_type == MODEL_POLYNOMIAL:
55             return np.column_stack((np.ones_like(x), x, x**2))
56         if model_type in (MODEL_SPLINE, MODEL_FUNCTIONAL):
57             columns = [np.ones_like(x), x, x**2, x**3]
58             columns.extend(np.maximum(x - knot, 0.0) ** 3 for knot in (0.25, 0.50, 0.75))
59             return np.column_stack(columns)
60         raise ValueError(f"unknown model type: {model_type}")
61
62
63     def candidate_configs(model_type: str) -> list[ModelConfig]:
64         if model_type == MODEL_POLYNOMIAL:
65             return [ModelConfig(value, 0.0) for value in (0.01, 0.1, 1.0, 10.0)]
66         if model_type == MODEL_SPLINE:
67             return [
68                 ModelConfig(random_penalty, curve_penalty)
69                 for random_penalty in (0.03, 0.3, 3.0)
70                 for curve_penalty in (0.001, 0.01, 0.1, 1.0)
71             ]
72         if model_type == MODEL_FUNCTIONAL:
73             return [ModelConfig(0.0, value) for value in (0.0001, 0.001, 0.01, 0.1, 1.0)]
74         raise ValueError(f"unknown model type: {model_type}")
75
76
77     def fit_population_model(data: pd.DataFrame, model_type: str, config: ModelConfig) -> PopulationCurveModel:
78         """Fit a strategy population curve with equal inferential units at battery level.
79
80         The two mixed candidates minimize
81
82             
$$\|y - F\beta - Zb\|^2 + \lambda_{\text{curve}} \|P\beta\|^2 + \lambda_{\text{random}} \|b\|^2,$$

83         where  $Z$  contains a random intercept and slope for each battery. The
84         functional candidate first smooths each battery, then averages cell
85         coefficients within strategy so batteries receive equal weight.
86         """
87         required = {"battery_id", "cycle", "policy", "SOH_clean"}
88         missing = required.difference(data.columns)

```

```

89     if missing:
90         raise ValueError(f"missing columns: {sorted(missing)}")
91     frame = data.loc[np.isfinite(data["SOH_clean"]) & np.isfinite(data["cycle"])]
92     frame["policy"] = frame["policy"].astype(str)
93     policy_names = tuple(pd.unique(frame["policy"]).tolist())
94     battery_ids = pd.unique(frame["battery_id"]).astype(int)
95     policy_index = {name: i for i, name in enumerate(policy_names)}
96     battery_index = {int(value): i for i, value in enumerate(battery_ids)}
97     basis = make_basis(model_type, frame["cycle"].to_numpy())
98     n_rows, n_basis = basis.shape
99
100    if model_type in (MODEL_POLYNOMIAL, MODEL_SPLINE):
101        fixed = np.zeros((n_rows, len(policy_names) * n_basis), dtype=float)
102        pid = frame["policy"].map(policy_index).to_numpy(dtype=int)
103        rows = np.arange(n_rows)
104        for j in range(n_basis):
105            fixed[rows, pid * n_basis + j] = basis[:, j]
106
107        random = np.zeros((n_rows, 2 * len(battery_ids)), dtype=float)
108        bid = frame["battery_id"].map(battery_index).to_numpy(dtype=int)
109        x = frame["cycle"].to_numpy(dtype=float) / 200.0
110        random[rows, 2 * bid] = 1.0
111        random[rows, 2 * bid + 1] = x
112        design = np.column_stack((fixed, random))
113
114        penalty = np.zeros(design.shape[1], dtype=float)
115        if model_type == MODEL_SPLINE:
116            for p in range(len(policy_names)):
117                start = p * n_basis
118                penalty[start + 2 : start + n_basis] = config.lambda_curve
119        penalty[len(policy_names) * n_basis :] = config.lambda_random
120        coefficient = _penalized_solve(design, frame["SOH_clean"].to_numpy(), penalty)
121        fixed_coef = coefficient[: len(policy_names) * n_basis].reshape(len(policy_names), n_basis)
122        random_coef = coefficient[len(policy_names) * n_basis :].reshape(len(battery_ids), 2)
123        return PopulationCurveModel(
124            model_type, config, policy_names, fixed_coef, battery_ids, random_coef=random_coef
125        )
126
127    if model_type == MODEL_FUNCTIONAL:
128        cell_coef = np.empty((len(battery_ids), n_basis), dtype=float)
129        cell_policy = np.empty(len(battery_ids), dtype=object)
130        penalty = np.zeros(n_basis, dtype=float)
131        penalty[2:] = config.lambda_curve
132        for i, battery_id in enumerate(battery_ids):
133            use = frame["battery_id"].to_numpy() == battery_id
134            cell_coef[i] = _penalized_solve(basis[use], frame.loc[use, "SOH_clean"].to_numpy(), penalty)
135            cell_policy[i] = frame.loc[use, "policy"].iloc[0]
136        fixed_coef = np.vstack([cell_coef[cell_policy == name].mean(axis=0) for name in policy_names])
137        return PopulationCurveModel(
138            model_type,
139            config,
140            policy_names,
141            fixed_coef,
142            battery_ids,
143            cell_coef=cell_coef,
144            cell_policy=cell_policy,
145        )
146    raise ValueError(f"unknown model type: {model_type}")
147
148
149    def _penalized_solve(design: np.ndarray, response: np.ndarray, penalty: np.ndarray) -> np.ndarray:
150        lhs = design.T @ design
151        scale = max(float(np.trace(lhs)) / max(lhs.shape[0], 1), 1.0)
152        lhs.flat[:, lhs.shape[0] + 1] += penalty + 1e-12 * scale

```

```

153     rhs = design.T @ response
154     try:
155         return np.linalg.solve(lhs, rhs)
156     except np.linalg.LinAlgError:
157         return np.linalg.lstsq(lhs, rhs, rcond=1e-12)[0]

```

```

1     """Shared numerical core for Question 2 smoke tests.
2
3     The strategy parameter coordinate, rather than a cycle row or a battery, is the
4     independent design unit. All parameter-response regressions therefore average
5     within strategy first and validate by leaving an entire parameter coordinate out.
6     """
7
8     from __future__ import annotations
9
10    from dataclasses import dataclass
11    from pathlib import Path
12
13    import numpy as np
14    import pandas as pd
15
16
17    SEED = 20260814
18    LAMBDA_GRID = (0.0, 0.01, 0.1, 1.0, 10.0, 100.0)
19    CHECKPOINTS = np.array([25, 50, 75, 100, 125, 150, 175, 200], dtype=int)
20
21
22    @dataclass(frozen=True)
23    class Candidate:
24        name: str
25        features: tuple[str, ...]
26        family: str = "ridge"
27        quadratic_cycle: bool = False
28
29
30    STRATEGY_CANDIDATES = (
31        Candidate("constant_mean", (), "constant"),
32        Candidate("nearest_coordinate", ("C1", "q", "C2"), "nearest"),
33        Candidate("ridge_raw_C1_q_C2", ("C1", "q", "C2")),
34        Candidate("ridge_stage_E1_E2", ("E1", "E2")),
35        Candidate("ridge_J", ("J",)),
36        Candidate("ridge_H", ("H",)),
37        Candidate("ridge_J_H", ("J", "H")),
38        Candidate("ridge_Jhigh50", ("J_high_50",)),
39        Candidate("ridge_Jhigh60", ("J_high_60",)),
40        Candidate("ridge_Jhigh70", ("J_high_70",)),
41    )
42
43
44    HIERARCHICAL_CANDIDATES = (
45        Candidate("hier_cycle_no_stress", (), "hierarchical", False),
46        Candidate("hier_cycle_J_linear", ("J",), "hierarchical", False),
47        Candidate("hier_cycle_H_linear", ("H",), "hierarchical", False),
48        Candidate("hier_cycle_J_H_linear", ("J", "H"), "hierarchical", False),
49        Candidate("hier_cycle_J_quadratic", ("J",), "hierarchical", True),
50    )
51
52
53    def load_clean_data(project_root: Path) -> tuple[pd.DataFrame, pd.DataFrame]:
54        cycle_path = project_root / "data" / "processed" / "q1_cleaned" / "cycle_train_clean.csv"
55        summary_path = project_root / "data" / "processed" / "q1_cleaned" / "battery_summary_clean.csv"
56        cycles = pd.read_csv(cycle_path)
57        summary = pd.read_csv(summary_path)

```

```

58     counts = cycles.groupby("battery_id", observed=True)["cycle"].nunique()
59     complete_ids = counts[counts.eq(200)].index
60     cycles = cycles[cycles["battery_id"].isin(complete_ids)].copy()
61     summary = summary[summary["battery_id"].isin(complete_ids)].copy()
62     if cycles["SOH_clean"].isna().any():
63         raise ValueError("SOH_clean contains missing values in the complete cohort")
64     return cycles, summary
65
66
67 def coordinate_id(frame: pd.DataFrame) -> pd.Series:
68     return (
69         frame["C1"].map(lambda value: f"{value:.3f}")
70         + "|"
71         + frame["q"].map(lambda value: f"{value:.3f}")
72         + "|"
73         + frame["C2"].map(lambda value: f"{value:.3f}")
74     )
75
76
77 def add_protocol_features(frame: pd.DataFrame) -> pd.DataFrame:
78     result = frame.copy()
79     result["q"] = result["Q1"] / 100.0
80     result["structure_batch"] = result["policy"].astype(str).str.contains("NEWSTRUCTURE").astype(int)
81     result["T0"] = 60.0 * (result["q"] / result["C1"] + (0.8 - result["q"]) / result["C2"])
82     result["E1"] = result["q"] * result["C1"]
83     result["E2"] = (0.8 - result["q"]) * result["C2"]
84     result["J"] = result["E1"] + result["E2"]
85     result["H"] = 0.5 * (
86         result["C1"] * result["q"] ** 2
87         + result["C2"] * (0.8**2 - result["q"] ** 2)
88     )
89     for threshold in (0.5, 0.6, 0.7):
90         result[f"J_high_{int(threshold * 100)}"] = np.where(
91             threshold < result["q"],
92             result["C1"] * (result["q"] - threshold)
93             + result["C2"] * (0.8 - result["q"]),
94             result["C2"] * (0.8 - threshold),
95         )
96     result["coordinate_id"] = coordinate_id(result)
97     return result
98
99
100 def battery_degradation_summary(cycles: pd.DataFrame, battery_meta: pd.DataFrame) -> pd.DataFrame:
101     records: list[dict[str, float | int | str]] = []
102     meta = battery_meta.set_index("battery_id")
103     for battery_id, group in cycles.groupby("battery_id", observed=True):
104         group = group.sort_values("cycle")
105         baseline = float(group.loc[group["cycle"].between(1, 5), "SOH_clean"].mean())
106         end_soh = float(group.loc[group["cycle"].between(196, 200), "SOH_clean"].mean())
107         x = group["cycle"].to_numpy(dtype=float) / 200.0
108         degradation = 1.0 - group["SOH_clean"].to_numpy(dtype=float) / baseline
109         design = np.column_stack((np.ones_like(x), x, x**2))
110         coef = np.linalg.lstsq(design, degradation, rcond=None)[0]
111         row = meta.loc[battery_id]
112         records.append(
113             {
114                 "battery_id": int(battery_id),
115                 "policy": str(row["policy"]),
116                 "C1": float(row["C1"]) if np.isfinite(row["C1"]) else np.nan,
117                 "Q1": float(row["Q1"]),
118                 "C2": float(row["C2"]),
119                 "baseline_soh": baseline,
120                 "soh200": end_soh,
121                 "relative_loss200": 1.0 - end_soh / baseline,

```

```

122         "curve_loss200": float(coef.sum()),
123         "curve_linear": float(coef[1]),
124         "curve_quadratic": float(coef[2]),
125         "mean_chargetime": float(row["mean_chargetime"]),
126         "mean_IR": float(row["mean_IR"]),
127         "mean_Tavg": float(row["mean_Tavg"]),
128         "flag_battery41": int(battery_id == 41),
129     }
130 )
131 return add_protocol_features(pd.DataFrame.from_records(records))
132
133
134 def strategy_summary(battery: pd.DataFrame) -> pd.DataFrame:
135     complete = battery[battery["C1"].notna()].copy()
136     feature_columns = [
137         "C1", "Q1", "C2", "q", "T0", "E1", "E2", "J", "H",
138         "J_high_50", "J_high_60", "J_high_70", "structure_batch",
139     ]
140     response_columns = [
141         "soh200", "relative_loss200", "curve_loss200", "curve_linear", "curve_quadratic",
142         "mean_chargetime", "mean_IR", "mean_Tavg",
143     ]
144     aggregations: dict[str, str | tuple[str, str]] = {column: "first" for column in feature_columns}
145     aggregations.update({column: "mean" for column in response_columns})
146     aggregations["n_batteries"] = ("battery_id", "size")
147     result = complete.groupby("policy", as_index=False, observed=True).agg(**{
148         key: value if isinstance(value, tuple) else (key, value) for key, value in aggregations.items()
149     })
150     result["coordinate_id"] = coordinate_id(result)
151     result["equal_time_cohort"] = result["policy"].ne("3_6C-80PER_3_6C").astype(int)
152     result["explicit_new_structure_cohort"] = result["structure_batch"].eq(1).astype(int)
153     return result.sort_values("policy").reset_index(drop=True)
154
155
156 def standardized(train: pd.DataFrame, test: pd.DataFrame, features: tuple[str, ...]) -> tuple[np.ndarray, np.ndarray, np.ndarray,
157     np.ndarray]:
158     if not features:
159         return np.empty((len(train), 0)), np.empty((len(test), 0)), np.array([], np.array([]))
160     mean = train.loc[:, features].mean(axis=0).to_numpy(dtype=float, copy=True)
161     scale = train.loc[:, features].std(axis=0, ddof=0).to_numpy(dtype=float, copy=True)
162     scale[scale < 1e-12] = 1.0
163     return (
164         (train.loc[:, features].to_numpy(dtype=float) - mean) / scale,
165         (test.loc[:, features].to_numpy(dtype=float) - mean) / scale,
166         mean,
167         scale,
168     )
169
170 def ridge_fit(x: np.ndarray, y: np.ndarray, ridge_lambda: float) -> np.ndarray:
171     design = np.column_stack((np.ones(len(x)), x))
172     penalty = np.eye(design.shape[1]) * ridge_lambda
173     penalty[0, 0] = 0.0
174     lhs = design.T @ design + penalty
175     return np.linalg.lstsq(lhs, design.T @ y, rcond=None)[0]
176
177
178 def ridge_predict(coef: np.ndarray, x: np.ndarray) -> np.ndarray:
179     return np.column_stack((np.ones(len(x)), x)) @ coef
180
181
182 def select_lambda_inner(train: pd.DataFrame, response: str, features: tuple[str, ...]) -> float:
183     groups = train["coordinate_id"].unique().tolist()
184     if len(groups) <= 3:

```

```

185     return 10.0
186 scores: list[tuple[float, float]] = []
187 for ridge_lambda in LAMBDA_GRID:
188     errors: list[float] = []
189     for group in groups:
190         inner_train = train[train["coordinate_id"].ne(group)]
191         inner_test = train[train["coordinate_id"].eq(group)]
192         x_train, x_test, _, _ = standardized(inner_train, inner_test, features)
193         coef = ridge_fit(x_train, inner_train[response].to_numpy(dtype=float), ridge_lambda)
194         prediction = ridge_predict(coef, x_test)
195         errors.append(float(np.mean((prediction - inner_test[response].to_numpy(dtype=float)) ** 2)))
196     scores.append((float(np.mean(errors)), ridge_lambda))
197 minimum = min(value for value, _ in scores)
198 tolerance = minimum * 1.01 + 1e-15
199 return max(ridge_lambda for value, ridge_lambda in scores if value <= tolerance)
200
201
202 def nearest_prediction(train: pd.DataFrame, test: pd.DataFrame, response: str) -> np.ndarray:
203     features = ("C1", "q", "C2")
204     x_train, x_test, _, _ = standardized(train, test, features)
205     y = train[response].to_numpy(dtype=float)
206     output = np.empty(len(test), dtype=float)
207     for index, row in enumerate(x_test):
208         distance = np.sum((x_train - row) ** 2, axis=1)
209         output[index] = y[int(np.argmin(distance))]
210     return output
211
212
213 def fit_hierarchical_penalized(
214     train_cycles: pd.DataFrame,
215     train_strategy: pd.DataFrame,
216     features: tuple[str, ...],
217     quadratic: bool,
218     lambda_fixed: float = 1.0,
219     lambda_battery: float = 1.0,
220     lambda_policy: float = 10.0,
221 ) -> tuple[np.ndarray, dict[str, object]]:
222     """Fit a linearized smoke surrogate of Model C by penalized least squares.
223
224     
$$y_{it} = \beta_0 + \beta_x x + \sum \beta_k z_{pk} x + \beta_q x^2 + a_i + u_i x + v_p x + \text{error}.$$

225
226     For runtime, this smoke fit uses cycle 1 and every fifth cycle. The formal
227     nonlinear exponential-link/AR(1) likelihood is intentionally not claimed
228     here. This surrogate answers whether the additional hierarchy has enough
229     predictive signal to justify a later full Model C fit.
230
231     """
232     strategy_lookup = train_strategy.set_index("policy")
233     frame = train_cycles[
234         train_cycles["policy"].isin(strategy_lookup.index)
235         & (train_cycles["cycle"].eq(1) | train_cycles["cycle"].mod(5).eq(0))
236     ].copy()
237     policies = sorted(frame["policy"].unique().tolist())
238     batteries = sorted(frame["battery_id"].unique().tolist())
239     p_index = {name: index for index, name in enumerate(policies)}
240     b_index = {value: index for index, value in enumerate(batteries)}
241     means = train_strategy.loc[:, features].mean(axis=0).to_numpy(dtype=float, copy=True) if features else np.array([])
242     scales = train_strategy.loc[:, features].std(axis=0, ddof=0).to_numpy(dtype=float, copy=True) if features else np.array([])
243     if len(scales):
244         scales[scales < 1e-12] = 1.0
245     x = frame["cycle"].to_numpy(dtype=float) / 200.0
246     fixed_columns = [np.ones(len(frame)), x]
247     for feature, mean, scale in zip(features, means, scales):
248         z = frame["policy"].map(strategy_lookup[feature]).to_numpy(dtype=float)

```

```

249     fixed_columns.append(((z - mean) / scale) * x)
250 if quadratic:
251     fixed_columns.append(x**2)
252 fixed = np.column_stack(fixed_columns)
253 random_battery = np.zeros((len(frame), 2 * len(batteries)))
254 random_policy = np.zeros((len(frame), len(policies)))
255 rows = np.arange(len(frame))
256 bidx = frame["battery_id"].map(b_index).to_numpy(dtype=int)
257 pidx = frame["policy"].map(p_index).to_numpy(dtype=int)
258 random_battery[rows, 2 * bidx] = 1.0
259 random_battery[rows, 2 * bidx + 1] = x
260 random_policy[rows, pidx] = x
261 design = np.column_stack((fixed, random_battery, random_policy))
262 penalty = np.zeros(design.shape[1])
263 penalty[2 : 2 + len(features)] = lambda_fixed
264 penalty[fixed.shape[1] : fixed.shape[1] + random_battery.shape[1]] = lambda_battery
265 penalty[fixed.shape[1] + random_battery.shape[1] :] = lambda_policy
266 lhs = design.T @ design
267 lhs.flat[:: lhs.shape[0] + 1] += penalty + 1e-10
268 rhs = design.T @ frame["SOH_clean"].to_numpy(dtype=float)
269 try:
270     coef = np.linalg.solve(lhs, rhs)
271 except np.linalg.LinAlgError:
272     coef = np.linalg.lstsq(lhs, rhs, rcond=None)[0]
273 fitted = design @ coef
274 residual = frame["SOH_clean"].to_numpy(dtype=float) - fitted
275 residual_frame = pd.DataFrame({"battery_id": frame["battery_id"].to_numpy(), "cycle": frame["cycle"].to_numpy(), "residual":
    residual})
276 pairs = []
277 for _, group in residual_frame.groupby("battery_id", observed=True):
278     values = group.sort_values("cycle")["residual"].to_numpy()
279     if len(values) > 1:
280         pairs.append(np.corrcoef(values[:-1], values[1:])[0, 1])
281 info = {
282     "fixed_coef": coef[: fixed.shape[1]],
283     "feature_mean": means,
284     "feature_scale": scales,
285     "features": features,
286     "quadratic": quadratic,
287     "train_rmse": float(np.sqrt(np.mean(residual**2))),
288     "lag1_residual_correlation": float(np.nanmean(pairs)),
289     "n_policy": len(policies),
290     "n_battery": len(batteries),
291 }
292 return coef, info
293
294
295 def predict_hierarchical(info: dict[str, object], test_strategy: pd.DataFrame, cycles: np.ndarray) -> np.ndarray:
296     features = tuple(info["features"])
297     means = np.asarray(info["feature_mean"], dtype=float)
298     scales = np.asarray(info["feature_scale"], dtype=float)
299     fixed_coef = np.asarray(info["fixed_coef"], dtype=float)
300     predictions: list[dict[str, float | int | str]] = []
301     x = cycles.astype(float) / 200.0
302     for _, row in test_strategy.iterrows():
303         columns = [np.ones(len(x)), x]
304         for feature, mean, scale in zip(features, means, scales):
305             columns.append(np.repeat((float(row[feature]) - mean) / scale, len(x)) * x)
306         if bool(info["quadratic"]):
307             columns.append(x**2)
308         value = np.column_stack(columns) @ fixed_coef
309         for cycle, prediction in zip(cycles, value):
310             predictions.append({"policy": row["policy"], "cycle": int(cycle), "prediction": float(prediction)})
311     return pd.DataFrame(predictions)

```

```

1  """Data loading, trajectory utilities, metrics, and constrained extrapolation."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7  from typing import Iterable
8
9  import numpy as np
10 import pandas as pd
11
12 from .config import CONFIG, Q3Config
13
14
15 @dataclass
16 class BatteryRecord:
17     battery_id: int
18     policy: str
19     meta: pd.Series
20     cycles: pd.DataFrame
21     baseline: float
22     relative_soh: np.ndarray
23
24     def relative_at(self, cycle: int) -> float:
25         return float(self.relative_soh[cycle - 1])
26
27     def absolute_future(self, start: int = 151, end: int = 200) -> np.ndarray:
28         return self.baseline * self.relative_soh[start - 1 : end]
29
30
31 def load_records(project_root: Path) -> tuple[dict[int, BatteryRecord], pd.DataFrame, pd.DataFrame]:
32     data_dir = project_root / "data" / "processed" / "q1_cleaned"
33     meta = pd.read_csv(data_dir / "battery_summary_clean.csv")
34     cycles = pd.read_csv(data_dir / "cycle_train_clean.csv")
35     records: dict[int, BatteryRecord] = {}
36     for battery_id, frame in cycles.groupby("battery_id", sort=True):
37         row = meta.loc[meta["battery_id"].eq(battery_id)].iloc[0]
38         frame = frame.sort_values("cycle").reset_index(drop=True)
39         baseline = float(frame.loc[frame["cycle"].between(1, 5), "SOH_clean"].mean())
40         records[int(battery_id)] = BatteryRecord(
41             battery_id=int(battery_id),
42             policy=str(row["policy"]),
43             meta=row,
44             cycles=frame,
45             baseline=baseline,
46             relative_soh=frame["SOH_clean"].to_numpy(float) / baseline,
47         )
48     return records, meta, cycles
49
50
51 def complete_battery_ids(meta: pd.DataFrame) -> list[int]:
52     return sorted(meta.loc[meta["prediction_test"].eq(0), "battery_id"].astype(int).tolist())
53
54
55 def slope(values: np.ndarray, cycles: np.ndarray | None = None) -> float:
56     values = np.asarray(values, dtype=float)
57     if cycles is None:
58         cycles = np.arange(1, len(values) + 1, dtype=float)
59     else:
60         cycles = np.asarray(cycles, dtype=float)
61     good = np.isfinite(values) & np.isfinite(cycles)
62     if good.sum() < 2:
63         return 0.0
64     x = cycles[good]

```



```

65     y = values[good]
66     xc = x - x.mean()
67     denom = float(xc @ xc)
68     return 0.0 if denom <= 0 else float(xc @ (y - y.mean()) / denom)
69
70
71 def robust_slope_scale(slopes: Iterable[float]) -> float:
72     values = np.asarray(list(slopes), dtype=float)
73     values = values[np.isfinite(values)]
74     if values.size == 0:
75         return 1e-8
76     median = float(np.median(values))
77     scale = 1.4826 * float(np.median(np.abs(values - median)))
78     if scale < 1e-8:
79         scale = max(float(np.std(values, ddof=1)) if values.size > 1 else 0.0, 1e-8)
80     return scale
81
82
83 def fit_power_law(
84     cycles: np.ndarray,
85     relative_soh: np.ndarray,
86     config: Q3Config = CONFIG,
87 ) -> dict[str, float]:
88     t = np.asarray(cycles, dtype=float)
89     y = np.asarray(relative_soh, dtype=float)
90     good = np.isfinite(t) & np.isfinite(y)
91     t, y = t[good], y[good]
92     if t.size < 5:
93         return {"beta0": float(np.nanmean(y)), "a": 0.0, "p": 1.0, "sse": np.inf}
94     L = float(t.max())
95     weights = 1.0 + 2.0 * (t / L) ** 2
96     root_w = np.sqrt(weights)
97     best: dict[str, float] | None = None
98     for p in config.power_grid:
99         z = t**p
100         design = np.column_stack([np.ones_like(t), -z])
101         coef, *_ = np.linalg.lstsq(design * root_w[:, None], y * root_w, rcond=None)
102         beta0, a = float(coef[0]), float(coef[1])
103         if a < 0:
104             a = 0.0
105             beta0 = float(np.average(y, weights=weights))
106         pred = beta0 - a * z
107         sse = float(np.sum(weights * (y - pred) ** 2))
108         candidate = {"beta0": beta0, "a": a, "p": float(p), "sse": sse}
109         if best is None or sse < best["sse"]:
110             best = candidate
111     assert best is not None
112     return best
113
114
115 def predict_power_law(fit: dict[str, float], cycles: np.ndarray) -> np.ndarray:
116     t = np.asarray(cycles, dtype=float)
117     return fit["beta0"] - fit["a"] * t ** fit["p"]
118
119
120 def power_law_eol(fit: dict[str, float], baseline: float, config: Q3Config = CONFIG) -> tuple[float, str]:
121     threshold = 0.8 / baseline
122     beta0, a, p = fit["beta0"], fit["a"], fit["p"]
123     if not np.isfinite([beta0, a, p]).all() or a <= 0 or beta0 <= threshold or p <= 0:
124         return np.nan, "no_finite_intersection"
125     cycle = float(((beta0 - threshold) / a) ** (1.0 / p))
126     if not np.isfinite(cycle) or cycle > config.eol_max_cycle:
127         return np.nan, "beyond_5000"
128     if cycle <= 150:

```

```

129     return cycle, "before_or_at_observation"
130     return cycle, "finite_scenario"
131
132
133 def project_absolute_prediction(raw: np.ndarray, anchor: float, config: Q3Config = CONFIG) -> np.ndarray:
134     clipped = np.clip(np.asarray(raw, dtype=float), *config.soh_bounds)
135     return np.minimum.accumulate(np.concatenate([[float(anchor)], clipped]))[1:]
136
137
138 def prediction_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
139     residual = np.asarray(y_pred) - np.asarray(y_true)
140     return {
141         "rmse": float(np.sqrt(np.mean(residual**2))),
142         "mae": float(np.mean(np.abs(residual))),
143         "error_cycle200": float(residual[-1]),
144     }
145
146
147 def strategy_parameters(record: BatteryRecord) -> np.ndarray:
148     row = record.meta
149     return np.asarray([row.get("C1", np.nan), row.get("Q1", np.nan), row.get("C2", np.nan)], dtype=float)

```

```

1  """Auditable Q4 discrete Pareto and constrained single-exposure models."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11  from q3_models.core import load_records
12
13
14  Q4_VERSION = "q4_smoke_v1"
15  SEED = 20260815
16  LAMBDA_GRID = np.array([0.0, 0.25, 0.5, 0.75, 1.0])
17
18
19  @dataclass(frozen=True)
20  class PolicyObservation:
21      policy: str
22      n_battery: int
23      time_mean: float
24      time_sd: float
25      loss_mean: float
26      loss_sd: float
27      late_slope_mean: float
28      c1: float
29      q1: float
30      c2: float
31      j: float
32      h: float
33      coordinate: tuple[float, float, float] | None
34
35
36  def _finite_mean(values: pd.Series) -> float:
37      values = pd.to_numeric(values, errors="coerce").dropna()
38      return float(values.mean()) if len(values) else float("nan")
39
40
41  def collect_policy_observations(project_root: Path) -> tuple[list[PolicyObservation], pd.DataFrame]:

```

```

42 records, meta, _ = load_records(project_root)
43 complete_ids = set(meta.loc[meta["prediction_test"].eq(0), "battery_id"].astype(int))
44 rows = []
45 observations: list[PolicyObservation] = []
46 for policy, group in meta.loc[meta["battery_id"].isin(complete_ids)].groupby("policy", sort=True):
47     battery_rows = []
48     for battery_id in sorted(group["battery_id"].astype(int)):
49         record = records[battery_id]
50         cycles = record.cycles
51         time_column = "chargetime_raw" if "chargetime_raw" in cycles.columns else "chargetime"
52         if time_column not in cycles.columns:
53             raise KeyError("Q4 requires chargetime_raw (or legacy chargetime) in cleaned cycle data")
54         time_series = pd.to_numeric(cycles[time_column], errors="coerce")
55         time_value = float(time_series.mean())
56         loss_value = float(1.0 - record.relative_at(200))
57         late_values = record.relative_soh[150:200]
58         x = np.arange(151, 201, dtype=float)
59         xc = x - x.mean()
60         slope = float(xc @ (late_values - late_values.mean()) / (xc @ xc))
61         battery_rows.append({"battery_id": battery_id, "time": time_value,
62                             "loss": loss_value, "late_slope_loss": max(0.0, -slope)})
63     values = pd.DataFrame(battery_rows)
64     first = group.iloc[0]
65     c1, q1, c2 = (float(first[col]) if pd.notna(first[col]) else np.nan for col in ("C1", "Q1", "C2"))
66     q = q1 / 100.0 if np.isfinite(q1) else np.nan
67     j = q * c1 + (0.8 - q) * c2 if np.isfinite([c1, q, c2]).all() else np.nan
68     h = 0.5 * (c1 * q**2 + c2 * (0.8**2 - q**2)) if np.isfinite([c1, q, c2]).all() else np.nan
69     coordinate = (c1, q1, c2) if np.isfinite([c1, q1, c2]).all() else None
70     observations.append(PolicyObservation(
71         policy=str(policy), n_battery=len(values), time_mean=float(values["time"].mean()),
72         time_sd=float(values["time"].std(ddof=1)) if len(values) > 1 else 0.0,
73         loss_mean=float(values["loss"].mean()),
74         loss_sd=float(values["loss"].std(ddof=1)) if len(values) > 1 else 0.0,
75         late_slope_mean=float(values["late_slope_loss"].mean()),
76         c1=c1, q1=q1, c2=c2, j=j, h=h, coordinate=coordinate,
77     ))
78     for row in battery_rows:
79         row.update({"policy": str(policy), "c1": c1, "q1": q1, "c2": c2, "j": j, "h": h})
80     rows.append(row)
81 return observations, pd.DataFrame(rows)
82
83
84 def observation_frame(observations: list[PolicyObservation]) -> pd.DataFrame:
85     return pd.DataFrame([obs.__dict__ for obs in observations])
86
87
88 def _normalise(values: np.ndarray) -> np.ndarray:
89     values = np.asarray(values, dtype=float)
90     lo, hi = np.nanmin(values), np.nanmax(values)
91     return np.zeros_like(values) if hi - lo < 1e-12 else (values - lo) / (hi - lo)
92
93
94 def pareto_mask(time: np.ndarray, loss: np.ndarray) -> np.ndarray:
95     time, loss = np.asarray(time, float), np.asarray(loss, float)
96     mask = np.ones(len(time), dtype=bool)
97     for i in range(len(time)):
98         dominated = (time <= time[i] + 1e-12) & (loss <= loss[i] + 1e-12)
99         strictly = (time < time[i] - 1e-12) | (loss < loss[i] - 1e-12)
100         mask[i] = not bool(np.any(dominated & strictly))
101     return mask
102
103
104 def choose_scalar(time: np.ndarray, loss: np.ndarray, policies: list[str], weight: float) -> int:
105     score = weight * _normalise(time) + (1.0 - weight) * _normalise(loss)

```

```

106     order = sorted(range(len(score)), key=lambda i: (score[i], loss[i], time[i], policies[i]))
107     return order[0]
108
109
110 def bootstrap_pareto(
111     frame: pd.DataFrame,
112     repetitions: int = 2000,
113     seed: int = SEED,
114     lambda_grid: np.ndarray | None = None,
115     loss_limits: tuple[float, ...] = (),
116 ) -> pd.DataFrame:
117     rng = np.random.default_rng(seed)
118     lambda_grid = LAMBDA_GRID if lambda_grid is None else np.asarray(lambda_grid, dtype=float)
119     policies = sorted(frame["policy"].unique())
120     records = {policy: frame.loc[frame["policy"].eq(policy)].reset_index(drop=True) for policy in policies}
121     rows = []
122     for rep in range(repetitions):
123         summary = []
124         for policy in policies:
125             data = records[policy]
126             idx = rng.integers(0, len(data), size=len(data))
127             sample = data.iloc[idx]
128             late_slope = float(sample["late_slope_loss"].mean()) if "late_slope_loss" in sample else np.nan
129             summary.append((policy, float(sample["time"].mean()), float(sample["loss"].mean()), late_slope))
130         t = np.array([row[1] for row in summary]); d = np.array([row[2] for row in summary])
131         front = pareto_mask(t, d)
132         choices = {w: summary[choose_scalar(t, d, policies, w)][0] for w in lambda_grid}
133         constrained_choices = {}
134         for limit in loss_limits:
135             feasible = np.flatnonzero(d <= limit)
136             constrained_choices[limit] = (
137                 min(feasible, key=lambda i: (t[i], d[i], policies[i])) if len(feasible) else None
138             )
139         for index, ((policy, time_value, loss_value, late_slope), is_front) in enumerate(zip(summary, front)):
140             rows.append({"version": Q4_VERSION, "replicate": rep, "policy": policy,
141                         "time": time_value, "loss": loss_value, "late_slope_loss": late_slope,
142                         "pareto": bool(is_front),
143                         **{f"selected_lambda_{w:.2f}": choices[w] == policy for w in lambda_grid},
144                         **{f"selected_loss_limit_{limit:.4f}": constrained_choices[limit] == index
145                            for limit in loss_limits}})
146     return pd.DataFrame(rows)
147
148
149 def fit_single_exposure(train: pd.DataFrame, exposure: str, lam: float) -> tuple[float, float, float, float]:
150     loss_column = "loss" if "loss" in train.columns else "loss_mean"
151     train = train.dropna(subset=[exposure, loss_column])
152     if len(train) < 2 or train[exposure].std(ddof=0) < 1e-8:
153         return float(train[loss_column].mean()), 0.0, float(train[exposure].mean()), 1.0
154     mean, scale = float(train[exposure].mean()), float(train[exposure].std(ddof=0))
155     z = (train[exposure].to_numpy(float) - mean) / scale
156     x = np.column_stack([np.ones(len(z)), z]); y = train[loss_column].to_numpy(float)
157     penalty = np.diag([0.0, lam]); coef = np.linalg.solve(x.T @ x + penalty, x.T @ y)
158     return float(coef[0]), float(coef[1]), mean, scale
159
160
161 def loso_single_exposure(
162     frame: pd.DataFrame,
163     exposure: str = "j",
164     ridge_grid: tuple[float, ...] = (0.01, 0.1, 1.0, 10.0),
165 ) -> pd.DataFrame:
166     usable = frame.dropna(subset=[exposure, "coordinate", "loss_mean"]).copy()
167     usable["coordinate_key"] = usable["coordinate"].astype(str)
168     rows = []
169     for held_out in sorted(usable["coordinate_key"].unique()):

```

```

170 train = usable.loc[usable["coordinate_key"].ne(held_out)]
171 test = usable.loc[usable["coordinate_key"].eq(held_out)]
172 best = None
173 for lam in ridge_grid:
174     intercept, slope, mean, scale = fit_single_exposure(train, exposure, lam)
175     if len(test) == 0 or scale < 1e-8:
176         pred = np.full(len(test), intercept)
177     else:
178         pred = intercept + slope * (test[exposure].to_numpy(float) - mean) / scale
179         rmse = float(np.sqrt(np.mean((pred - test["loss_mean"].to_numpy(float)) ** 2))) if len(test) else np.nan
180         candidate = (rmse, -lam, intercept, slope, mean, scale)
181         if best is None or candidate[:2] < best[:2]: best = candidate
182     assert best is not None
183     baseline_rmse = float(np.sqrt(np.mean((test["loss_mean"].to_numpy(float) - train["loss_mean"].mean()) ** 2))) if len(test)
184         else np.nan
185     rows.append({"version": Q4_VERSION, "exposure": exposure, "held_out_coordinate": held_out,
186                 "n_test_policy": len(test), "rmse": best[0], "lambda": -best[1],
187                 "constant_rmse": baseline_rmse, "improvement": baseline_rmse - best[0],
188                 "intercept": best[2], "slope": best[3], "train_mean": best[4],
189                 "train_scale": best[5], "validation_type": "coordinate_LOSO_extrapolation_pressure"})
190 return pd.DataFrame(rows)

```

附录 C AI 工具使用详情

本参赛队在竞赛中使用 AI 工具辅助完成以下环节：

1. 代码调试与运行环境配置；
2. 语言润色；
3. 数据处理与建模方案讨论。

核心建模思路、模型选择、数值结果核验与最终结论均由参赛队独立完成，AI 生成的代码与文字均已逐项人工审查与核实。详细交互过程见支撑材料”AI 工具使用详情.pdf”。